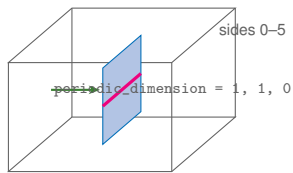
3D SIMULATIONS

What changes in three dimensions: the input file's six sides, structures as sheets and bristles, the cost, and the 3D examples of this series



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 12

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. write a 3D input file: three numbers wherever 2D had two, six sides numbered 0 to 5 in the boundary-condition blocks, a third velocity component with its own block, and a domain that need not be a cube;
2. build a 3D structure file set (points with $x\ y\ z$, springs and beams along two directions of a sheet, target points along a line) and move its target points in 3D, as the course's leaf and moving plates did;
3. run the 3D examples of this series, IB/08 (a bristle in a sheared box, target points) and IBFE/10 (a plate in a flow tank, finite elements), and say what each shows;
4. estimate what a 3D run costs in cells, steps and cores, and why adaptive refinement and a short first run are not optional;
5. find the 3D output (three numbers per point in `hier_data_IB3d`; `viz_IB3d` for VisIt) and know where chapter 27 takes it.

> Nothing in the IB method changes between 2D and 3D; the delta function has a third factor and every array a third component. What changes is the cost, and the number of places where a third number has to be right.

HOW THIS CHAPTER WAS CHECKED

- The two 3D examples (IB/08 glycocalyx, IBFE/10 flow-tank plate) build and run with IBAMR 0.16.0 and 0.19.0; their READMEs record the numbers quoted here. The side numbering and the input keys are those of the current `muParserRobinBcCoefs` and `CartesianGeometry` input blocks, as read by those runs.
- The 2016 lecture *3D Simulations* (public folder 8) is quoted in full; its 3D leaf example was a Google Drive link that no longer resolves, so the sheet is described from the slides and IB/08 stands in for the hands-on part.
- The 2012 lab ReadMe deck (the `moving_plates2d`/`moving_plates3d` clap-and-fling model, kinematics in `update_target_point_positions.C`, the wing's four files and the generator run twice) is quoted; the 3D wing files and the lab's 2014 wing-mesh scripts (Ellington's planform) exist in the lab folders and are described, not shipped.
- The 2013 *Math769 3D* deck in the course folders is in a format that no current tool can open; nothing from it is claimed here.
- A student's 2012 "how to use 3D IBAMR" deck is a walk-through of ssh, emacs and `screen` on a lab machine; its one technical line (`mpirun -np 4 ./main3d input3d`) is in chapter 1's terms already.

> Slide numbers refer to this deck; chapter numbers (1, 5, 6, 7, 9a, 11, 16, 18, 22, 27, 28) to the other chapters of this series.

WHERE TO LOOK THINGS UP

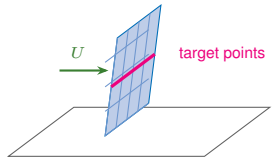
- The examples: `examples/IB/08-glycocalyx-3d` (`input3d`, `generate_mesh3d.py/.m`, `update_target_point_positions.cpp`, README with the shear-flow timing) and `examples/IBFE/10-flow-tank-plate-3d` (`input3d`, `plate2.msh`, README); chapter 22 for the latter's physics.
- Upstream 3D examples: `examples/IB/explicit/ex7` (3D posts held by target points, IB/08's relative); `examples/IBFE/explicit/ex4` and `ex5` in 3D (IBFE/10's relatives); chapter 15's tour.
- The input blocks: chapter 7 (boundary conditions) for the Robin form and the `VelocityBcCoefs_d` blocks; this chapter for the six sides and the third component.
- Structure files in 3D: chapter 6 (the formats are the same with three coordinates); IB/08's generator for a cylinder of rings; chapter 18 for finite-element meshes (gmsh).
- Visualization: chapter 27 (VisIt in 3D: slices, isosurfaces, the 2D slice from a 3D run) and chapter 28 (finite-element data).
- The IBAMR documentation, https://ibamr.github.io/docs; the FAQ, https://github.com/IBAMR/IBAMR/wiki/FAQ.

THE COURSE'S GOAL: A 3D LEAF

From the 2016 lecture *IBAMR boundary conditions / 3D simulations* (public folder 8): "Our goal will be to develop a 3D simulation of a flexible 'leaf'. The leaf will be tethered to target points along its midline and immersed in flow."

"Rectangular sheet in flow. In this case the leaf is simply a rectangle constructed from beams and springs. Target points along the centerline hold it in place. The matlab file `generate_mesh3d.m` makes the necessary files."

The 2016 slides showed the sheet twice: a rendering of the bent sheet in the stream, and the generator's picture of the flat rectangle of points with the centreline marked. The example's files were on a Google Drive link that no longer resolves; the sheet is rebuilt in words on slide 11, and IB/08 (slide 15) is the 3D example you can run.



The 2016 sheet, redrawn: a rectangle of points joined by springs and beams along both directions, held by a line of target points along its middle, in a stream.

REVIEW: ROBIN BOUNDARY CONDITIONS

"Robin boundary conditions are a weighted combination of Dirichlet boundary conditions and Neumann boundary conditions. If Ω is the domain on which the given equation is to be solved and $\partial\Omega$ denotes its boundary, the Robin boundary condition is

$$a\,u + b\,\frac{\partial u}{\partial n} = g \quad \text{on } \partial\Omega$$

for some non-zero constants a and b and a given function g defined on $\partial\Omega$." (2016)

Chapter 7 has the whole story: IBAMR's `acoef`, `bcoef` and `gcoef` functions are these a, b and g, one set per velocity component and per side, and IBAMR's solver reads only the two pairs $(a, b) = (1, 0)$ (a value) and $(0, 1)$ (a normal derivative); anything in between is not a boundary condition it can apply. In 3D nothing changes but the count: three components, six sides.

IBAMR'S BOUNDARY NUMBERING IN 3D

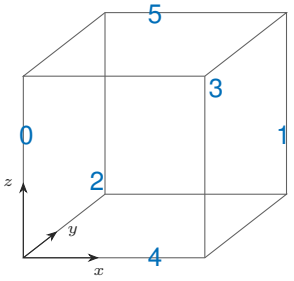
"Look in the `input3d` file.

- 2D: `xleft = 0`, `xright = 1`, `ybottom = 2`, `ytop = 3`
- 3D: add `zbottom = 4`, `ztop = 5`." (2016)

The rule behind the numbers: side $2d$ is the lower end of direction d and side $2d+1$ the upper end ($d = 0, 1, 2$ for x, y, z). The 2016 slide drew the box with the six numbers on its faces; the sketch here is the same.

```
// one block per velocity component, six functions of each kind
VelocityBcCoefs_2 {                     // the z-component (IB/08)
   acoef_function_0 = "1.0"             // side 0: x lower
   acoef_function_1 = "1.0"             // side 1: x upper
   acoef_function_2 = "1.0"             // side 2: y lower
   acoef_function_3 = "1.0"             // side 3: y upper
   acoef_function_4 = "1.0"             // side 4: z lower (the floor)

   acoef_function_5 = "1.0"             // side 5: z upper (the lid)
   bcoef_function_0 = "0.0"  ...    bcoef_function_5 = "0.0"
   gcoef_function_0 = "0.0"  ...    gcoef_function_5 = "0.0"
}
```



Sides 0–5 of a 3D box. A block with only `_0` to `_3` functions leaves sides 4 and 5 undefined, and the run stops at start-up with a message naming the missing key.

NO SLIP IN 3D, AND A BACKGROUND FLOW

"No slip in 3D. See the `input3d` file. Note that `VelocityBcCoefs_0` would set the x-component of velocity, and `VelocityBcCoefs_2` would set the z-component of velocity." (2016) The slide showed `VelocityBcCoefs_1` with all six `acoef` $= 1$, `bcoef` $= 0$, `gcoef` $= 0$: the y-velocity is zero on every side.

"Background flow in the x-direction. In this case we set all of the a's to one. The x-component of the velocity should be set to the desired background flow (0.1 in this case). The y- and z-components should be set to zero." (2016)

```
VelocityBcCoefs_0 {                          // boundary conditions
  for the x-component of the velocity
   acoef_function_0 = "1.0"   ...  acoef_function_5 = "1.0"
   bcoef_function_0 = "0.0"   ...  bcoef_function_5 = "0.0"
   gcoef_function_0 = "0.1"   ...  gcoef_function_5 = "0.1"
}
```

"Initial conditions. Previously these have all been set to zero. With the nonzero boundary conditions, we need to change these to the background flow rate." (2016)

```
VelocityInitialConditions {
   function_0 = "0.1"    // the x-component
   function_1 = "0.0"    // the y-component
   function_2 = "0.0"    // the z-component
}
```

A third `function` line, and the same value as the boundaries' g. An initial field of 0 against boundaries at 0.1 is the impulsive start of chapter 7, in 3D; the 2016 example did it the other way round, a uniform 0.1 everywhere, which starts the fluid moving through the leaf at $t = 0$.

Periodic in a direction: the `VelocityBcCoefs` entries for its two sides are ignored (next slide), but they must still be present for the file to parse.

“IMPROVEMENTS”: THE RAMP THE COURSE PROMISED

“One negative effect of starting with a nonzero background flow is that this has the effect of making a leaf suddenly appear in flow. The result is that very large forces will be generated initially. A possible solution to this (coming soon) would be to start at $u = 0$ and ramp up the velocity smoothly to some nonzero value.” (2016)

Delivered, in every example of this series that has a stream: the boundaries carry `U_FLOW*tanh(2*t)` (IB/03, chapter 7), the lid of IB/08 moves as `V*tanh(2*t)`, IBFE/10’s six sides carry `U*tanh(2*t)`, and the initial velocity is zero. The ramp reaches 96% of its final value at $t = 1$ s; a faster one (`tanh(10*t)`) is fine for a stiff structure, a slower one for a floppy sheet.

In 3D the impulsive start is worse than in 2D for a plain reason: the first step’s pressure impulse acts on the whole sheet at once, and a sheet of N^2 points has N times the force of a filament of N points on the same tethers. Chapter 11’s force files show the spike; chapter 16 shows the abort it can cause.

DOMAIN SIZE AND PERIODIC BOUNDARIES IN 3D

"Look at `input3d`. `x_lo` sets xmin, ymin and zmin; `x_up` sets xmax, ymax and zmax. `Periodic_dimension` turns on periodic boundary conditions and overrides everything. Set this to zero to turn off periodic BC's." (2016)

```
CartesianGeometry {
   domain_boxes = [ (0,0,0) , (N - 1, N - 1, N - 1) ]
   x_lo = -0.5*L, -0.5*L, -0.5*L      // lower end of
  computational domain
   x_up =  0.5*L,  0.5*L,  0.5*L      // upper end of
  computational domain
   periodic_dimension = 1, 1, 1
}
```

The 2016 file as shown: a cube of N^3 coarse cells, periodic in all three directions.

What the 2016 input did not say:

- with `periodic_dimension = 1, 1, 1` the `VelocityBcCoefs` blocks of the previous slides, inflow included, are ignored: a periodic box has no sides to apply them to. The 2016 leaf therefore saw a uniform initial velocity in a periodic box, not an inflow; chapter 7's rule ("periodic overrides everything") is the same in 3D. For a stream, set the stream's direction to 0 (IBFE/10: `0, 0, 0`; IB/08: `1, 1, 0`, periodic across, walls above and below);
- the box need not be a cube: IB/08 is $1/8 \times 1/8 \times 1$ with `domain_boxes = [(0,0,0), (N/8-1, N/8-1, N-1)]`; the coarse cells are cubes, so the counts follow the lengths;
- three numbers everywhere: `x_lo`, `x_up`, `periodic_dimension`, `domain_boxes`, the `ratio_to_coarser` and patch-size lines of the gridding block, and the `largest_patch_size` that limits memory per process.

CONSTRUCTING A SHEET: SPRINGS AND BEAMS IN TWO DIRECTIONS

```
# a rectangular sheet of nx by ny points in the plane x = x0 (the 2016 leaf, rebuilt)
# point (i, j) has number i + nx*j; y = y0 + i*ds, z = z0 + j*ds
with open(f"{name}.vertex", "w", newline="\n") as f:
    f.write("%d\n" % (nx * ny))
    for j in range(ny):
        for i in range(nx):
            f.write("%1.16e %1.16e %1.16e\n" % (x0, y0 + i * ds, z0 + j * ds))
# springs along rows and along columns: (nx-1)*ny + nx*(ny-1) of them
# beams along rows and along columns: (nx-2)*ny + nx*(ny-2) of them
for j in range(ny):
    for i in range(nx - 1):  spring(i + nx*j, i + 1 + nx*j)
    for i in range(nx - 2):  beam(i + nx*j, i + 1 + nx*j, i + 2 + nx*j)
for i in range(nx):
    for j in range(ny - 1):  spring(i + nx*j, i + nx*(j + 1))
    for j in range(ny - 2):  beam(i + nx*j, i + nx*(j + 1), i + nx*(j + 2))
# target points along the centreline j = ny//2
for i in range(nx):  target(i + nx*(ny // 2))
```

The same three file types as in 2D (chapter 6), with three coordinates per vertex line and the same spring, beam and target lines; the stiffnesses in the files are the 2D ones per unit width, i.e. $K\,\Delta s/\Delta s$ for springs and $k_b\,\Delta s/\Delta s^3$ for beams with the sheet's own spacing. IB/08's generator builds a cylinder the same way (rings of points, `x y z` per line, every point a target). What a sheet needs that a filament did not is on the next slide.

A SHEET IS NOT A FILAMENT: SHEAR, TWIST, AND THE COUNT OF POINTS

Three things a sheet needs that a filament did not:

- shear: springs along rows and columns alone let the rectangle shear into a parallelogram; add diagonal springs (from (i, j) to $(i+1, j+1)$ and $(i+1, j)$ to $(i, j+1)$) if the sheet should keep its angles;
- bending in two directions: beams along rows resist one curvature, beams along columns the other; a sheet that must also resist twist needs the diagonals' beams too;
- the point spacing: $\Delta s \approx \Delta x/2$ of the finest level in *both* directions, so $N_{\text{points}} \sim (L/\Delta s)^2$: a 0.1 m square leaf at $\Delta x = 1/512$ m is $103 \times 103 = 10{,}609$ points, and the structure files are a hundred times longer than the leaf's.

> The stiffness bookkeeping of chapter 6 applies per link: a spring along a row of a sheet with spacing Δs carries $K/\Delta s$ in the file like any other, and the sheet's stiffness per unit width comes out of how many rows there are. The 2016 generator used the leaf's 2D values unchanged, which is right for a sheet of the same spacing.

MOVING TARGET POINTS IN 3D

"Update target points. Another option would be to set the initial and boundary conditions to zero and translate the wing. This could be done with `update_target_point_positions`. Just change `V` from zero to some value." (2016)

```
if (leaf_lag_idxs.first <= lag_idx && lag_idx < leaf_lag_idxs.
  second)
{
    X_target[0] += V*dt;
}
```

Chapter 9a's first form (move the target by $V\Delta t$ each step), with `X_target` now a 3-vector: `[0]`, `[1]`, `[2]` for x, y, z. IB/08 ships exactly this (`update_target_point_positions.cpp`, called only when `TARGET_VELOCITY` is not 0): the bristle can be dragged through the fluid instead of the lid dragging the fluid past it.

From the 2012 lab ReadMe: "Simple clap and fling. `Moving_plates` is a simple proto clap-and-fling IBAMR model. All it does is translate the left wing to the left and the right wing to the right. You all will need to add in the more complex kinematics. When you do so, you may also need to play around with the timestep sizes. You know you have a timestep that is too big when you get an error quickly or when the results don't make sense (very noisy). Timesteps are set by the parameter `dt` in the input files (`input2d` or `input3d`)."

"The kinematics of the wings are specified in a C++ function defined in the file `update_target_point_positions.C`. If you modify either `main.C` or `update_target_point_positions.[Ch]`, you will need to recompile the program."

IB/07 is the 2D version of that model, with its own README; the 3D one is the same code with three components and the 3D wing files of the next slide.

THE 2012 MOVING_PLATES3D: FILES, GENERATOR, RUNNING

"Parameters. The model parameters are defined in a couple of different files. Fluid parameters are defined in the input files (`input2d` for the 2D model, `input3d` for the 3D model). The material parameters are stored in the files:

```
plate3d_left_512.vertex     <=== initial
  positions of the vertices
plate3d_left_512.spring     <=== list of springs
  in the model
plate3d_left_512.beam       <=== list of beams in
   the model
plate3d_left_512.target     <=== list of target
  points in the model
```

These files are generated by a simple Matlab script, where various material parameters are defined. 2D and 3D versions of the Matlab script are in the `moving_plates` folder (`generate_mesh3d.m` and `generate_mesh2d.m`). Run the matlab function twice (setting the if statement to 0 and 1) to make the left wing and the right wing." (2012)

"3D code. For the three-dimensional case, go to `.../examples/moving_plates3d`. Next you can do `make clean`, `make cleandist`, `make main3d`, `./main3d input3d`." And on the cluster of the day: "To run a 3D simulation, type `make main3d` and `bsub -q week -n 16 mpirun ./main3d input3d`. `-n 16` will run on 16 processors. I recommend running 16–32 (should be powers of 2)." (2012)

Today: `cmake` with `IBAMR::IBAMR3d` builds `main3d` (IB/08's `CMakeLists.txt`), and chapter 1's `job.slurm` replaces `bsub`. The wing of 2012 was a $4{,}661$-point sheet (`plate3d_left_512.vertex`); the lab's 2014 scripts generalized it to Ellington's beta-function planform (`square_wing.m`, `beta_wing_mesh.m`: chord, span, the shape parameters $\hat{r}_1$, $\hat{r}_2$, an angle of attack), which chapter 20's wings use. "Run the function twice" is chapter 9a's two-structure pattern: one file set per wing, both named in `structure_names`.

IB/08, THE 3D IB EXAMPLE OF THIS SERIES: A BRISTLE IN A SHEARED BOX

From its README: “One glycocalyx fiber, a cylinder made of 13,600 target points, stands on the floor of a box of fluid. The lid of the box slides sideways, so the fluid is sheared past the fiber. The box is periodic in x and y, so the run models one fiber in an endless square array of fibers. It shows a box that is not a cube, periodic and wall boundaries together, a moving wall, target points in 3D, and 3D output for VisIt.”

- Box $1/8 \times 1/8 \times 1$ m, periodic in x and y; the floor fixed, the lid moving at $V \tanh(2t)$, $V = 0.1$ m/s.
- Grid: 3 levels, ratio 4, $4 \times 4 \times 32$ coarse cells, $\Delta x = 1/512$ m on the finest level, which covers only $40 \times 40 \times 96$ cells around the fiber; $161{,}280$ cells in all.
- Fluid $\rho = 1000$, $\mu = 40$; $\mathrm{Re} = 0.061$ on the fiber’s height.
- Fiber: radius 0.0132 m, height 0.156 m, 160 rings of 85 points, every point a target with $\kappa = 3.19 \times 10^4$.
- 20,000 steps of 10^{-5} s, about 20 minutes on 4 cores.

What its README teaches that a 2D run could not:

- how long the flow takes to arrive: the lid’s motion diffuses down with $\nu = 0.04$; the 1D solution gives 0.5% of the final shear at the fiber’s top at 1 s, 55% at 4 s, 96% at 10 s. The course ran 10 s, a million steps; the default here stops at 0.2 s, before the flow reaches the fiber, and `SHEAR_START = 1` starts from the steady shear instead;
- a 3D stiffness trap: the target stiffness in the file is κ/ds with the generator’s $ds =$ radius$/84$, not the points’ true spacing around the ring (6.2 times larger); kept so that the files match every earlier version, and flagged;
- 3D output: `viz_IB3d`, `hier_data_IB3d` with three numbers per point, the fiber as a mesh in VisIt (chapter 27).

IB/08's INPUT3D: WHAT DIFFERS FROM 2D, LINE BY LINE

```
N = 32                              // coarse cells in z; x and y have N/8
...
CartesianGeometry {
   domain_boxes = [ (0,0,0) , (N/8 - 1, N/8 - 1, N - 1) ]
   x_lo = -L/16, -L/16, -L/2        // lower end of computational domain
   x_up =  L/16,  L/16,  L/2        // upper end of computational domain
   periodic_dimension = 1, 1, 0     // periodic across, walls above and below
}
GriddingAlgorithm {
   ratio_to_coarser { level_1 = REF_RATIO, REF_RATIO, REF_RATIO ... }
   largest_patch_size { level_0 = 512, 512, 512 }
   smallest_patch_size { level_0 = 8, 8, 8 }
   ...
}
VelocityBcCoefs_0 { ... six acoef, six bcoef, six gcoef; gcoef_function_5 = "V*tanh(2*t)" }
VelocityBcCoefs_1 { ... }
VelocityBcCoefs_2 { ... }           // the block 2D does not have
VelocityInitialConditions { function_0 = "0.0"  function_1 = "0.0"  function_2 = "0.0" }
```

Read it on the next slide.

IB/08's INPUT3D, READ

- `domain_boxes` with three indices; the counts in x and y are an eighth of the count in z because the box is;
- `x_lo`, `x_up`, `periodic_dimension` with three entries;
- every `ratio_to_coarser`, `largest_patch_size`, `smallest_patch_size` line with three entries;
- a third `VelocityBcCoefs` block, each block with six sides; the lid's motion is `gcoef_function_5` of the x-component (side 5 is z upper);
- a third `function` in the initial conditions;
- the program is built as `main3d` (`IBAMR::IBAMR3d`); `NDIM` is 3 in every loop of `main.cpp` and the update function, with no edits.

The muParser functions may use `X_0`, `X_1`, `X_2` and `t`; a lid velocity that varies across the lid (`V*tanh(2*t)*(1 - 4*X_1*X_1/(L*L))`) is one line.

IBFE/10, THE 3D IBFE EXAMPLE: A PLATE IN A FLOW TANK

From its README: “A thin elastic plate stands across a uniform stream in a 3D flow tank. A band across the middle of the plate is held in place by a tether (target) force, and the parts above and below the band are free to bend with the flow. It reads a tetrahedral mesh made with gmsh, uses a compressible neo-Hookean material, applies a tether force to part of the body, and writes the plate’s motion to an ExodusII file.”

- Tank $32 \times 16 \times 16$, not periodic; six sides at $(U \tanh(2t), 0, 0)$ with $U = 0.1$; $\rho = 1$, $\mu = 0.01$, $\mathrm{Re} = 20$ on the plate’s width.
- Grid: 3 levels, ratio 4, $32 \times 16 \times 16$ coarse cells, $\Delta x = 1/16$ on the finest level around the plate.
- Plate 0.125 thick, 2×2, 3,386 nodes and 11,481 tetrahedra; 5,000 steps, 30–50 minutes on 4 cores.

It is the 3D version of the 2016 leaf with a different structure model: the sheet of springs and beams becomes a solid with a stress law (chapter 19), the line of target points a tethered band (chapter 22), and the `.vertex` file a gmsh mesh (chapter 18). The input file’s fluid half is the one on the previous slide with `periodic_dimension = 0, 0, 0` and a stream on all six sides.

WHAT 3D COSTS

- Cells: a uniform grid of N cells per side has N^3: $512^3 = 1.3 \times 10^8$, against $512^2 = 2.6 \times 10^5$ in 2D, five hundred times more. Adaptive refinement is what makes 3D possible: IB/08's finest level covers $40 \times 40 \times 96 = 1.5 \times 10^5$ cells of a $64 \times 64 \times 512$ box, and the whole hierarchy 1.6×10^5.
- Points: a sheet has $(L/\Delta s)^2$ points, a surface $(L/\Delta s)^2$, a volume $(L/\Delta s)^3$ (IBFE); IB/08's bristle, a surface, has $13{,}600$.
- Steps: the time step limit is the same as in 2D (chapter 5), set by Δx and the stiffness, so a run of the same physical time has the same number of steps; each step costs the number of cells and points. IB/08: 0.05 s per step on 4 cores for 1.6×10^5 cells; the course's 10 s were a million steps, a day on 16 cores.
- Memory: every cell carries velocity, pressure, the tagging and the forcing; a million cells is a few hundred MB per field set. `largest_patch_size` bounds what one process holds; a 3D run is what the 2012 advice "16–32 processors, powers of 2" and Puma's nodes (chapter 1) are for.
- Output: a `viz_IB3d` frame of 10^6 cells is tens of MB; 50 frames of IB/08 are manageable, 500 are not. Write `hier_data` often and the fields rarely (chapter 11).

> The first 3D run should be short (IB/08's 0.2 s), on the coarsest grid that resolves the structure, with one frame written: check the geometry, the sides, the ramp and the step before spending a day of cluster time. Then chapter 16's convergence test.

3D OUTPUT AND WHERE IT GOES NEXT

- `hier_data_IB3d/X.NNNNN` and `F.NNNNN`: three numbers per point, read with `ndim = 3` by chapter 11's readers; the lab's 2014 3D script summed `A(1:3:end)`, `A(2:3:end)`, `A(3:3:end)`.
- `viz_IB3d/dumps.visit`: the fields on the 3D hierarchy; `lag_data.visit`: the structures as 3D point meshes (IB/08's `glyco_512_vertices`; a `_mesh` of edges exists only for structures with springs, chapter 28).
- `IB3d.log` and the `integral{F}` lines with three components.
- The IBFE examples write the solid to an ExodusII file (`.ex2`) with libMesh (chapter 28).

Chapter 27 is the VisIt chapter for 3D: a slice through the box (the 2015 note *Taking a 2D slice from 3D IBAMR with VisIt*), isosurfaces of vorticity magnitude, a mesh plot of the structure with the slice behind it, and the camera. The 2012 ReadMe's advice still holds: "open up the `lag_data.visit` database, make a 'mesh' plot: you should be able to make a line plot of the springs and a point plot of the vertices; `dumps.visit` plots the adaptive mesh, velocities, vorticities, etc."

WHAT CAN GO WRONG

The mistake	What happens
`periodic_dimension = 1, 1, 1` with an inflow	the inflow is ignored; the fluid moves only if the initial condition says so (the 2016 input). Set the stream's direction to 0
a `VelocityBcCoefs` block with four sides	the run stops at start-up asking for `acoef_function_4`; copy all six
no `VelocityBcCoefs_2`	the same, for the z-component
two numbers in `x_lo` or `domain_boxes`	a parse error, or a box with the wrong shape
initial velocity $\neq$ boundary velocity	chapter 7's impulsive start; use the ramp on the boundary and zero inside
Δs from the wrong length (IB/08's radius$/84$)	a stiffness several times off; compute the spacing from neighbouring points
a sheet without diagonals or cross beams	it shears or twists freely; add them if the physics needs them
a 2D time step in 3D	the same Δx allows the same Δt; but a finer Δx chosen to fit memory does not
the course's 10 s on a laptop	a million steps; run 0.2 s first, then the cluster
one process for a million cells	memory; use 16–32 processes and `largest_patch_size`

HOMEWORK

From the 2012–2016 courses and this chapter (Appendix C collects the full set):

1. IB/08: run the default 0.2 s, then `SHEAR_START = 1`, and compare the force on the fiber (chapter 11, `ndim = 3`) at the end of each.
2. IB/08: drag the fiber instead of the lid (`TARGET_VELOCITY`), with the lid at rest; is the force the same?
3. Build the 2016 sheet with slide 11's recipe (0.1 m square, $\Delta s = \Delta x/2$ at $\Delta x = 1/256$ m), tether its centreline, put it in IB/08's box with a stream in x (`periodic_dimension = 0, 1, 0`), and run 0.5 s. Report the number of points, cells and the time per step.
4. Add diagonal springs to the sheet and compare the shape at 0.5 s.
5. IBFE/10: halve the plate's shear modulus and compare the bending (chapter 22).

> What to hand in: the `input3d` blocks you changed, the generator loop, the point and cell counts, the time per step on the cores you used, and one VisIt image with a slice and the structure.

Extension: the two wings of the 2012 model, translated apart in 3D (IB/07's update function with `X_target[0]`), on a coarse grid.

RULES OF THUMB

- Three numbers wherever 2D had two; six sides, $2d$ lower and $2d + 1$ upper; a third component block.
- Periodic overrides the sides: a stream needs its direction non-periodic.
- Ramp the boundary, start the fluid at rest.
- A sheet is rows and columns of springs and beams, plus diagonals if it must not shear.
- $\Delta s \approx \Delta x/2$ in every direction of the structure; points grow as $(L/\Delta s)^2$.
- Refine only around the structure; check the cell count in the log before the long run.
- Short first, coarse first, one frame first; then the cluster, 16–32 processes.
- Positions and forces have three numbers per point; VisIt needs a slice.

SUMMARY

1. A 3D IBAMR run is the 2D one with a third coordinate everywhere: in the domain and gridding blocks, in six-sided boundary-condition blocks for three components, in a third initial condition, and in structure files with $x\ y\ z$ per point. The method does not change.
2. The course's 3D leaf (a sheet of springs and beams tethered along its midline) and the lab's clap-and-fling plates (two wings translated apart) are the 2D examples of chapters 7 and 9a in 3D; the recipes for the sheet, the target motion and the two-structure input are given here, and IB/08 and IBFE/10 are the runs.
3. Periodic directions ignore the sides; a stream needs a non-periodic direction, a ramp and a fluid at rest.
4. The cost is the point: N^3 cells and $(L/\Delta s)^2$ points, the same number of steps as in 2D, so adaptive refinement, a short coarse first run and a cluster are part of the method.
5. The output has three numbers per point and needs a slice to be seen; chapters 11 and 27 do that.

Next, chapter 13: adding mass, where points get a mass of their own and fall under gravity.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The 2016 lecture's input fragments are quoted and completed: the side rule $(2d,\ 2d+1)$, the third component block, the three-entry lines of the gridding block, and the fact that its `periodic_dimension = 1, 1, 1` silently ignored the inflow it set.
- "Improvements (coming soon)" is delivered: every example with a stream ramps it.
- The sheet generator, whose file is gone, is rebuilt as a recipe with the shear and twist caveats; the 2012 moving-plates model is described from the lab ReadMe with its wing files and the lab's 2014 planform scripts named.
- IB/08 and IBFE/10 replace the unavailable examples, with measured run times and the shear-flow timing that explains the course's 10-second runs.
- New: the cost slide, the output slide, the what-can-go-wrong table, and the homework that builds the 2016 sheet.
- Left out: the 2013 3D deck (unreadable format), the 2012 ssh walk-through (chapter 1), and the lab machine names and credentials of the 2012 decks.

RECORDINGS (LISTED, NOT INCORPORATED)

The front matter's video list (chapter 0.1) has no recording of the 2016 3D lecture. The 3D material that was recorded is VisIt's, and belongs to chapter 27: `2-Video_Visualizing3Ddata.mp4` (2018-02-26, 258 MB) and its 2019 copy `Lecture 2-Video_Visualizing3Ddata.mp4` (2019-07-03), and the 2022 VisIt tutorial's `3D flow.mp4` (2023-07-25, 40 MB). The decks they go with (*VISIT in 3D with IBAMR*, 2018 and 2020; *Taking a 2D slice from 3D IBAMR with VisIt*, 2015) are chapter 27's sources.

REFERENCES

- This series: `examples/IB/08-glycocalyx-3d` and `examples/IBFE/10-flow-tank-plate-3d` (READMEs with the measured numbers); chapters 7, 9a, 11, 15, 18, 22, 27, 28.
- IBAMR: `examples/IB/explicit/ex7` (3D posts held by target points), `examples/IBFE/explicit/ex4`, `ex5` (3D elastic bodies in a stream); the input keys of `CartesianGeometry`, `GriddingAlgorithm` and `muParserRobinBcCoefs`, https://ibamr.github.io/docs.
- C. P. Ellington, *The aerodynamics of hovering insect flight. II. Morphological parameters*, Phil. Trans. R. Soc. B 305 (1984) 17–40 (the beta-function planform of the lab's wing scripts).
- Sources: *IBAMR boundary conditions / 3D simulations* (public folder 8, 2016, 12 slides); the lab's *IBAMR ReadMe* deck (2012; the moving-plates model, kinematics, parameters, 3D code, cluster runs); *Math769 3D* (2013; unreadable); the lab's 3D wing files and scripts (2012–2014).